1. System Design & Architecture

1.1 System Overview:

Our Smart Drive System is designed to automate file storage and organization by using real-time location data to sort files into folders named after the city where they are uploaded. This system is capable of handling both automatic and manual city name inputs to ensure robustness in case of API failure to fetch the actual location.

1.2 Components:

Frontend (Client Side)

- A web-based portal designed with HTML, JavaScript, CSS.
- Drag-and-drop functionality for file uploads.
- HTML Geolocation API [R1] integration for fetching latitude and longitude.

Backend (Server Side)

- Designed using PyCharm IDE [R2]
- Python Flask-based local host server for processing and routing requests.
- OpenCage Reverse Geocoding API [R3] integration to convert geolocation data to city names.
- Fallback mechanism to handle cases where the city name cannot be determined automatically or when user blocks location access.

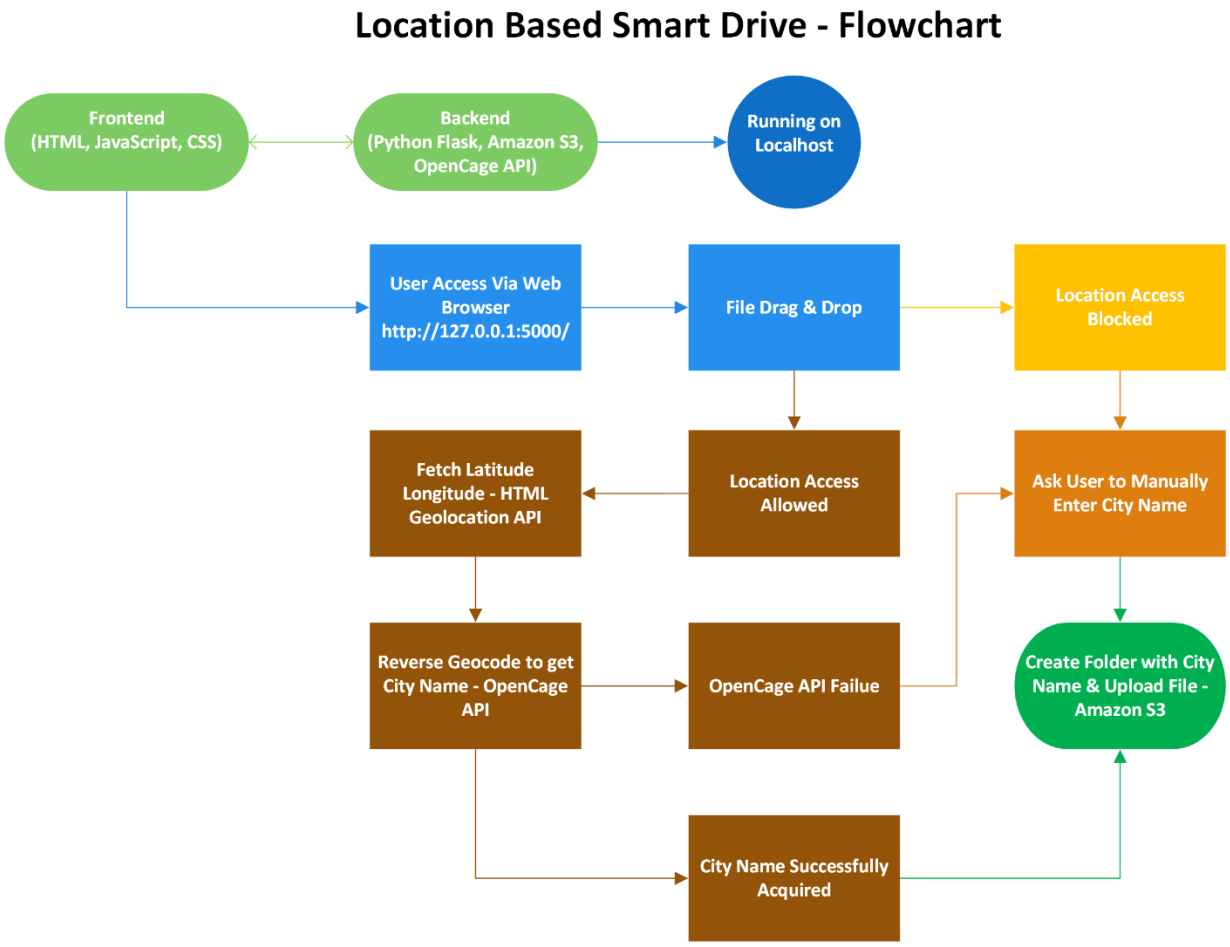
Cloud Storage (Amazon S3):

- Amazon S3 [R4] bucket for storing files organized into folders named after cities that are automatically acquired or manually entered by the user.

1.3 Data Flow:

1. Server is started on localhost setup using PyCharm IDE and can be accessed on any modern web browser (Google Chrome, Mozilla Firefox, Microsoft Edge) using address: http://127.0.0.1:5000/
2. User uploads files via the drag-and-drop interface.
3. The frontend prompts for location access and if user allows, it retrieves longitude and latitude using the HTML Geolocation API.
4. The backend processes the geolocation data and queries the OpenCage API for the city name.
5. If the API fails to retrieve city name, the user is prompted to manually input the city name and click “Upload File” to upload their file with manual city name.
6. Files are uploaded to the Amazon S3 bucket named “smart-drive-system” under a folder corresponding to the city name.

1.4 Architectural Diagram:



2. System Implementation Approach

2.1 Frontend Development

- Programming Languages: HTML, JavaScript, CSS.

Features:

- A user-friendly drag-and-drop interface.
- Integration with the HTML Geolocation API to obtain latitude and longitude.
- Location access prompt to user after dragging and dropping file on the interface.
- Feedback prompt to the user indicating success or failure of file upload.

2.2 Backend Development

- Framework: Flask (Python).

Modules, Libraries, APIs:

- **boto3** for Amazon S3 integration.
- **requests** for OpenCage API calls.
- OpenCage API: Fetches real-time latitude and longitude of the user using POST request and converts this data into city names.

Fallback Mechanism:

- If the OpenCage API fails to return a city name, the backend triggers a manual input request to the user.
- Ensures files are still organized effectively despite API issues.

2.3 Cloud Storage Integration:

Amazon S3:

- Bucket created to store files in folders named after city names.
- Bucket access is restricted. AWS access keys or root user access needed to view uploaded files and folders.
- Error handling embedded in code for cases where credentials or permissions might be missing.

2.4 Error Handling

- Frontend alerts users about API, upload failures or when location access is blocked.
- Backend returns appropriate error codes and messages for seamless debugging and user interaction.

2.5 Testing and Validation:

- Localhost setup to simulate real-world geolocation scenarios.
- Simulated API failures by intentionally entering wrong OpenCage API Key to validate the fallback mechanism.

Results:

- Automatic folder creation and upload to Amazon S3 worked as expected.
- Manual input fallback i.e. City Name worked when OpenCage API failed due to wrong key.

2.6 Deployment

- Hosted the backend server locally for testing. Accessed on any modern web browser (Google Chrome, Mozilla Firefox, Microsoft Edge) using address: http://127.0.0.1:5000/
- Configured Amazon S3, created bucket for scalable cloud storage.
- Used Flask's development server to validate functionality before full deployment.

2.7 Limitations

- Only accessible through localhost i.e. All the files must be uploaded from 1 computer on which backend is running.
- The system depends on the Geolocation API which can be inaccurate in some cases.
- Using free trial of OpenCage API which allows only 2500 requests per day.
- Using free trial of Amazon S3 which is valid for 12 months, allows 5 GB storage and 2000 PUT requests.
- System doesn't include security measures such as authentication or authorization for file uploads.

2.8 Recommendations for Future Enhancements

- Add user authentication for secure access to the portal.
- Use paid versions of APIs and Cloud Storage.
- Extend support for multiple file uploads.
- Integrate additional APIs for enhanced geolocation accuracy.
- Host backend on the cloud to enable access to portal from anywhere in the world.

REFERENCES

[R1] https://www.w3schools.com/html/html5_geolocation.asp

[R2] https://www.jetbrains.com/pycharm/

[R3] https://opencagedata.com/api

[R4] https://aws.amazon.com/s3/